

OJONUGWA WADA

May 25, 2025

0.0.1 Statistical Analysis of Workforce Engagement, Performance, and Compensation in HR Data

0.1 By

0.1.1 OJONUGWA WADA

0.1.2 Data Preprocessing and Description

STEP 1: DATA IMPORT AND INITIAL EXPLORATION

Purpose: This section loads the HR dataset and provides a quick preview to understand the structure, columns, and type of data we are dealing with. It helps lay the foundation for all further analysis.

```
[1]: # Import necessary libraries for data analysis and statistics
import pandas as pd # Used for data manipulation and analysis of tabular data
import numpy as np # Used for numerical operations, especially array/matrix
    ↪ computations
from scipy import stats # Used for statistical functions like mean, mode,
    ↪ standard error, etc.

# Load the HR dataset into a pandas DataFrame
file_path = 'HR_Dataset.csv' # Path to the dataset
df = pd.read_csv(file_path) # Read the dataset from CSV file

# Display a quick preview of the dataset
# This helps in understanding the columns, data types, and spotting any
    ↪ immediate issues (e.g., missing values)
print("Original Dataset Preview:")
display(df.head())
print(f"Shape of dataset: {df.shape}")
```

Original Dataset Preview:

	EmpID	MarriedID	MaritalStatusID	GenderID	EmpStatusID	DeptID	\
0	10026	0	0	1	1	5	
1	10084	1	1	1	5	3	
2	10196	1	1	0	5	5	
3	10088	1	1	0	1	5	
4	10069	0	2	0	5	5	

	PerfScoreID	FromDiversityJobFairID	Salary	Termd	...	ManagerName	\
0	4	0	62506	0	...	Michael Albert	
1	3	0	104437	1	...	Simon Roup	
2	3	0	64955	1	...	Kissy Sullivan	
3	3	0	64991	0	...	Elijah Gray	
4	3	0	50825	1	...	Webster Butler	

	ManagerID	RecruitmentSource	PerformanceScore	EngagementSurvey	\
0	22.0	LinkedIn	Exceeds	4.60	
1	4.0	Indeed	Fully Meets	4.96	
2	20.0	LinkedIn	Fully Meets	3.02	
3	16.0	Indeed	Fully Meets	4.84	
4	39.0	Google Search	Fully Meets	5.00	

	EmpSatisfaction	SpecialProjectsCount	LastPerformanceReview_Date	\
0	5	0	1/17/2019	
1	3	6	2/24/2016	
2	3	0	5/15/2012	
3	5	0	01/03/2019	
4	4	0	02/01/2016	

	DaysLateLast30	Absences
0	0	1
1	0	17
2	0	3
3	0	15
4	0	2

[5 rows x 35 columns]

Shape of dataset: (311, 35)

STEP 2: MISSING VALUE DETECTION

Purpose: In this step, we identify missing values in each column of the HR dataset. This is essential for data cleaning and ensures that our analysis and statistical methods are based on complete and reliable data.

```
[2]: # Print message to indicate that we're checking for missing values
print("\nMissing Values in Each Column:")

# Calculate the number of missing values in each column
missing_values = df.isnull().sum()

# Convert the result to a DataFrame for clean viewing, and sort by number of
↳ missing values in descending order
```

```
missing_values_df = missing_values.to_frame(name='Missing Values').
↳sort_values(by='Missing Values', ascending=False)

display(missing_values_df)
```

Missing Values in Each Column:

	Missing Values
DateofTermination	207
ManagerID	8
EmpID	0
DateofHire	0
TermReason	0
EmploymentStatus	0
Department	0
ManagerName	0
RecruitmentSource	0
HispanicLatino	0
PerformanceScore	0
EngagementSurvey	0
EmpSatisfaction	0
SpecialProjectsCount	0
LastPerformanceReview_Date	0
DaysLateLast30	0
RaceDesc	0
CitizenDesc	0
MarriedID	0
Salary	0
MaritalStatusID	0
GenderID	0
EmpStatusID	0
DeptID	0
PerfScoreID	0
FromDiversityJobFairID	0
Termd	0
MaritalDesc	0
PositionID	0
Position	0
State	0
Zip	0
DOB	0
Sex	0
Absences	0

STEP 3: HANDLING MISSING VALUES

Purpose: This step involves cleaning the dataset by addressing missing values. Certain columns are deemed critical (e.g., ManagerName, Position, Salary, DateofHire), and any row missing those values is dropped. For other numerical features, missing values are imputed with the median to prevent distortion from outliers.

Rationale:

- Dropping rows with missing critical values avoids incomplete or misleading records.
- Using the median instead of the mean provides a robust measure that isn't affected by extreme values.

```
[3]: # Define columns considered critical to our analysis-rows missing these will be removed
critical_columns = ['ManagerName', 'Position', 'Salary', 'DateofHire']
df_cleaned = df.dropna(subset=critical_columns)

# Define numerical columns where missing values will be imputed with the median
numerical_columns = [
    'Salary', 'EngagementSurvey', 'EmpSatisfaction',
    'SpecialProjectsCount', 'DaysLateLast30', 'Absences'
]

# Fill missing values in numerical columns using the median
for col in numerical_columns:
    if df_cleaned[col].isnull().sum() > 0:
        median_value = df_cleaned[col].median()
        df_cleaned[col].fillna(median_value, inplace=True)

# Preview the cleaned dataset
print("\nDataset After Handling Missing Values:")
display(df_cleaned.head())
```

Dataset After Handling Missing Values:

	EmpID	MarriedID	MaritalStatusID	GenderID	EmpStatusID	DeptID	\
0	10026	0	0	1	1	5	
1	10084	1	1	1	5	3	
2	10196	1	1	0	5	5	
3	10088	1	1	0	1	5	
4	10069	0	2	0	5	5	

	PerfScoreID	FromDiversityJobFairID	Salary	Termd	...	ManagerName	\
0	4	0	62506	0	...	Michael Albert	
1	3	0	104437	1	...	Simon Roup	
2	3	0	64955	1	...	Kissy Sullivan	
3	3	0	64991	0	...	Elijah Gray	
4	3	0	50825	1	...	Webster Butler	

	ManagerID	RecruitmentSource	PerformanceScore	EngagementSurvey	\
0	22.0	LinkedIn	Exceeds	4.60	
1	4.0	Indeed	Fully Meets	4.96	
2	20.0	LinkedIn	Fully Meets	3.02	
3	16.0	Indeed	Fully Meets	4.84	
4	39.0	Google Search	Fully Meets	5.00	

	EmpSatisfaction	SpecialProjectsCount	LastPerformanceReview_Date	\
0	5	0	1/17/2019	
1	3	6	2/24/2016	
2	3	0	5/15/2012	
3	5	0	01/03/2019	
4	4	0	02/01/2016	

	DaysLateLast30	Absences
0	0	1
1	0	17
2	0	3
3	0	15
4	0	2

[5 rows x 35 columns]

STEP 4: CLEANING TEXTUAL DATA (STRING COLUMNS)

Purpose: This step standardizes all textual (object-type) columns by removing leading/trailing whitespace and converting strings to title case. This prevents inconsistencies in categorical values caused by varied capitalization or hidden spaces.

Rationale: For example, 'sales', 'Sales', and 'Sales' should be treated the same. Standardizing text ensures better grouping, cleaner visuals, and more accurate filtering/aggregation later in analysis.

```
[5]: # Identify all columns with string (object) data types
string_columns = df_cleaned.select_dtypes(include=['object']).columns

# Clean each string column: strip leading/trailing whitespace only
for col in string_columns:
    df_cleaned[col] = df_cleaned[col].str.strip()

# Preview the cleaned string columns
print("\nDataset After Cleaning String Columns:")
display(df_cleaned[string_columns].head())
```

Dataset After Cleaning String Columns:

Position	State	DOB	Sex	MaritalDesc	CitizenDesc	\
----------	-------	-----	-----	-------------	-------------	---

0	Production Technician I	MA	07/10/1983	M	Single	US Citizen
1	Sr. DBA	MA	05/05/1975	M	Married	US Citizen
2	Production Technician II	MA	09/19/88	F	Married	US Citizen
3	Production Technician I	MA	09/27/88	F	Married	US Citizen
4	Production Technician I	MA	09/08/1989	F	Divorced	US Citizen

	HispanicLatino	RaceDesc	DateofHire	DateofTermination	TermReason	\
0	No	White	07/05/2011	NaN	N/A-StillEmployed	
1	No	White	3/30/2015	6/16/2016	career change	
2	No	White	07/05/2011	9/24/2012	hours	
3	No	White	01/07/2008	NaN	N/A-StillEmployed	
4	No	White	07/11/2011	09/06/2016	return to school	

	EmploymentStatus	Department	ManagerName	RecruitmentSource	\
0	Active	Production	Michael Albert	LinkedIn	
1	Voluntarily Terminated	IT/IS	Simon Roup	Indeed	
2	Voluntarily Terminated	Production	Kissy Sullivan	LinkedIn	
3	Active	Production	Elijah Gray	Indeed	
4	Voluntarily Terminated	Production	Webster Butler	Google Search	

	PerformanceScore	LastPerformanceReview_Date
0	Exceeds	1/17/2019
1	Fully Meets	2/24/2016
2	Fully Meets	5/15/2012
3	Fully Meets	01/03/2019
4	Fully Meets	02/01/2016

STEP 5: CONVERTING CATEGORICAL VARIABLES

Purpose: This step converts relevant string-based columns into the ‘category’ data type. Doing so reduces memory usage, speeds up processing, and enables certain types of analysis (e.g., using categorical variables in modeling or grouping operations).

Rationale: Categorical conversion is especially helpful for columns with repeated values (e.g., departments, race), as it tells pandas to treat them as labels rather than plain strings.

```
[6]: # Define columns that represent categorical attributes
categorical_columns = [
    'Sex', 'MaritalDesc', 'CitizenDesc', 'HispanicLatino', 'RaceDesc',
    'EmploymentStatus', 'Department', 'PerformanceScore'
]

# Convert each column to categorical data type
for col in categorical_columns:
    df_cleaned[col] = df_cleaned[col].astype('category')

# Show the updated data types for just the converted columns
```

```
print("\nDataset After Converting Categorical Variables:")
display(df_cleaned[categorical_columns].dtypes.to_frame(name='Data Types'))
```

Dataset After Converting Categorical Variables:

	Data Types
Sex	category
MaritalDesc	category
CitizenDesc	category
HispanicLatino	category
RaceDesc	category
EmploymentStatus	category
Department	category
PerformanceScore	category

STEP 6: CONVERTING DATE COLUMNS TO DATETIME FORMAT

Purpose: This step ensures that all date-related columns are correctly parsed as date-time objects. Accurate datetime formatting enables proper filtering, time-based calculations (like age, tenure), and avoids errors in further analysis.

Rationale: Using `errors='coerce'` ensures that any malformed or invalid dates are safely converted to NaT (Not a Time), which can then be handled appropriately (e.g., filtered or imputed).

```
[7]: # List columns expected to contain date information
date_columns = ['DOB', 'DateofHire', 'DateofTermination',
               ↪ 'LastPerformanceReview_Date']

# Convert each column to datetime format if it exists in the DataFrame
for col in date_columns:
    if col in df_cleaned.columns:
        df_cleaned[col] = pd.to_datetime(df_cleaned[col], errors='coerce')

# Preview only the columns that were actually converted
converted_columns = [col for col in date_columns if col in df_cleaned.columns]

print("\nDataset After Converting Date Columns:")
display(df_cleaned[converted_columns].head())
```

Dataset After Converting Date Columns:

	DOB	DateofHire	DateofTermination	LastPerformanceReview_Date
0	1983-07-10	2011-07-05	NaT	2019-01-17
1	1975-05-05	2015-03-30	2016-06-16	2016-02-24
2	1988-09-19	2011-07-05	2012-09-24	2012-05-15
3	1988-09-27	2008-01-07	NaT	2019-01-03
4	1989-09-08	2011-07-11	2016-09-06	2016-02-01

STEP 7: REMOVING DUPLICATE RECORDS

Purpose: This step checks for and removes duplicate rows to ensure each employee record is unique. Duplicates can skew statistical analysis and reporting, especially in headcounts or averages.

Rationale: Rather than removing blindly, we first check how many duplicate rows exist. If duplicates are found, we drop them and confirm the number of rows removed.

```
[8]: # Check how many duplicate rows exist (excluding the index)
num_duplicates = df_cleaned.duplicated().sum()

print(f"\nNumber of duplicate rows found: {num_duplicates}")

# Remove duplicates only if any are found
if num_duplicates > 0:
    df_cleaned = df_cleaned.drop_duplicates()
    print(f"{num_duplicates} duplicate rows have been removed.")
else:
    print("No duplicate rows found. No action taken.")

# Preview the cleaned dataset
print("\nDataset After Removing Duplicates:")
display(df_cleaned.head())
```

Number of duplicate rows found: 0

No duplicate rows found. No action taken.

Dataset After Removing Duplicates:

	EmpID	MarriedID	MaritalStatusID	GenderID	EmpStatusID	DeptID	\
0	10026	0	0	1	1	5	
1	10084	1	1	1	5	3	
2	10196	1	1	0	5	5	
3	10088	1	1	0	1	5	
4	10069	0	2	0	5	5	

	PerfScoreID	FromDiversityJobFairID	Salary	Termd	...	ManagerName	\
0	4		0 62506	0	...	Michael Albert	
1	3		0 104437	1	...	Simon Roup	
2	3		0 64955	1	...	Kissy Sullivan	
3	3		0 64991	0	...	Elijah Gray	
4	3		0 50825	1	...	Webster Butler	

	ManagerID	RecruitmentSource	PerformanceScore	EngagementSurvey	\
0	22.0	LinkedIn	Exceeds	4.60	
1	4.0	Indeed	Fully Meets	4.96	
2	20.0	LinkedIn	Fully Meets	3.02	

3	16.0	Indeed	Fully Meets	4.84
4	39.0	Google Search	Fully Meets	5.00

	EmpSatisfaction	SpecialProjectsCount	LastPerformanceReview_Date	\
0	5	0	2019-01-17	
1	3	6	2016-02-24	
2	3	0	2012-05-15	
3	5	0	2019-01-03	
4	4	0	2016-02-01	

	DaysLateLast30	Absences
0	0	1
1	0	17
2	0	3
3	0	15
4	0	2

[5 rows x 35 columns]

STEP 8: CREATING NEW FEATURES

Purpose: This step creates new features like ‘Age’ and ‘Tenure’ to offer insights into employee demographics and tenure, aiding HR decisions and lifecycle management.

Rationale: By calculating ‘Age’ from the birth year and ‘Tenure’ from the hire date, we derive useful features for analyzing employee age distribution and average tenure.

```
[9]: # Calculate the 'Age' of each employee by subtracting their birth year from the
      ↪ current year
current_year = pd.Timestamp.now().year
df_cleaned['Age'] = current_year - pd.DatetimeIndex(df_cleaned['DOB']).year

# Calculate the 'Tenure' of each employee by finding the difference between
      ↪ their hire date and the current date, then converting it to years
df_cleaned['Tenure'] = (pd.Timestamp.now() - df_cleaned['DateofHire']).dt.days /
      ↪ 365

# Display a preview of the dataset with the newly added 'Age' and 'Tenure'
      ↪ features
print("\nDataset After Adding New Features 'Age' and 'Tenure':")
display(df_cleaned[['EmpID', 'DOB', 'Age', 'DateofHire', 'Tenure']].head())
```

Dataset After Adding New Features 'Age' and 'Tenure':

	EmpID	DOB	Age	DateofHire	Tenure
0	10026	1983-07-10	42	2011-07-05	13
1	10084	1975-05-05	50	2015-03-30	10
2	10196	1988-09-19	37	2011-07-05	13

3	10088	1988-09-27	37	2008-01-07	17
4	10069	1989-09-08	36	2011-07-11	13

STEP 9: DROPPING UNNECESSARY COLUMNS

Purpose: This step removes non-essential columns, focusing the dataset on relevant features to improve processing efficiency.

Rationale: By defining the columns to keep and dropping others, we streamline the dataset, retaining only the important data for analysis.

```
[10]: # Define the columns that I want to keep in the dataset
columns_to_keep = [
    'EmpID', 'Salary', 'EngagementSurvey', 'EmpSatisfaction',
    'SpecialProjectsCount', 'DaysLateLast30', 'Absences',
    'PerformanceScore', 'Department', 'Position',
    'EmploymentStatus', 'DOB', 'Age',
    'DateofHire', 'Tenure'
]

# Create a list of columns to drop by selecting those not in the
↳ 'columns_to_keep' list
columns_to_drop = [col for col in df_cleaned.columns if col not in
↳ columns_to_keep]

# Remove the unnecessary columns from the dataset
# The 'errors='ignore'' argument ensures no error if a column is already missing
df_cleaned = df_cleaned.drop(columns=columns_to_drop, errors='ignore')

# Display a preview of the dataset after removing the unwanted columns
print("\nDataset After Dropping Unnecessary Columns:")
display(df_cleaned.head())
```

Dataset After Dropping Unnecessary Columns:

	EmpID	Salary	Position	DOB	DateofHire	\
0	10026	62506	Production Technician I	1983-07-10	2011-07-05	
1	10084	104437	Sr. DBA	1975-05-05	2015-03-30	
2	10196	64955	Production Technician II	1988-09-19	2011-07-05	
3	10088	64991	Production Technician I	1988-09-27	2008-01-07	
4	10069	50825	Production Technician I	1989-09-08	2011-07-11	

	EmploymentStatus	Department	PerformanceScore	EngagementSurvey	\
0	Active	Production	Exceeds	4.60	
1	Voluntarily Terminated	IT/IS	Fully Meets	4.96	
2	Voluntarily Terminated	Production	Fully Meets	3.02	
3	Active	Production	Fully Meets	4.84	
4	Voluntarily Terminated	Production	Fully Meets	5.00	

	EmpSatisfaction	SpecialProjectsCount	DaysLateLast30	Absences	Age	\
0	5	0	0	1	42	
1	3	6	0	17	50	
2	3	0	0	3	37	
3	5	0	0	15	37	
4	4	0	0	2	36	

	Tenure
0	13
1	10
2	13
3	17
4	13

STEP 10: SAVING THE CLEANED DATASET

Purpose: This step saves the cleaned dataset to a new CSV file for future use, ensuring the data is stored without needing to repeat the cleaning process.

Rationale: Saving the cleaned dataset preserves the final version for later analysis or reporting, maintaining data integrity and easy access.

```
[11]: # Specify the path where I want to save the cleaned dataset
cleaned_file_path = "HR_Dataset_Clean.csv"

# Write the cleaned dataset to the CSV file, making sure not to include the
# index column
df_cleaned.to_csv(cleaned_file_path, index=False)

# Print a message confirming that the cleaned dataset has been saved
print(f"\nCleaned dataset saved to {cleaned_file_path}")
```

Cleaned dataset saved to HR_Dataset_Clean.csv

STATISTICAL ANALYSIS SETUP: LOADING CLEANED HR DATA

Purpose: This section imports essential libraries and loads the pre-cleaned HR dataset to begin statistical analysis. The data is assumed to have gone through cleaning steps (missing values handled, formatting fixed, etc.)

```
[12]: # Import required libraries
import pandas as pd # For structured data manipulation
import numpy as np  # For numerical operations
from scipy import stats # For statistical analysis

# Load the cleaned HR dataset
cleaned_file_path = 'HR_Dataset_Clean.csv' # Path to the cleaned dataset
```

```

try:
    df = pd.read_csv(cleaned_file_path)
    print("Cleaned HR dataset loaded successfully.")
    print(f"Shape of dataset: {df.shape}")
except FileNotFoundError:
    print(f"Error: File '{cleaned_file_path}' not found.")
except Exception as e:
    print(f"An error occurred while loading the dataset: {e}")

```

Cleaned HR dataset loaded successfully.
Shape of dataset: (311, 15)

0.1.3 Task 1: Measure of Central Tendency

SCENARIO 1: MEAN SALARY BY DEPARTMENT

Purpose: This analysis calculates the average (mean) salary across different departments. It helps HR and management evaluate how compensation is distributed and whether certain departments are underpaid or overpaid relative to others.

Rationale: Understanding salary distribution by department can uncover disparities, inform compensation strategies, and assist with budgeting and workforce planning.

```

[13]: # Group the dataset by 'Department' and compute the mean salary
mean_salary_by_dept = df.groupby('Department', as_index=False)['Salary'].mean()

# Rename the column for clarity
mean_salary_by_dept.rename(columns={'Salary': 'Mean Salary'}, inplace=True)

# Sort the result for better readability
mean_salary_by_dept.sort_values(by='Mean Salary', ascending=False, inplace=True)

# Print output
display(
    mean_salary_by_dept.style
    .set_caption("Mean Salary by Department")
    .format({'Mean Salary': '{:,.2f}'))
)

```

<pandas.io.formats.style.Styler at 0x1c0f1a4f690>

SCENARIO 2: MEDIAN AGE BY PERFORMANCE SCORE

Purpose: This analysis calculates the median age of employees within each performance score category. It gives insight into generational trends across different performance levels.

Rationale:Median is used (instead of mean) because it's resistant to outliers and better reflects the 'typical' employee age. Identifying age patterns in high or low performers can help with training, mentoring, and workforce planning.

```
[14]: # Group by performance score and calculate the median age
median_age_by_perf = df.groupby('PerformanceScore', as_index=False)['Age'].
    ↪median()

# Rename the column for clarity
median_age_by_perf.rename(columns={'Age': 'Median Age'}, inplace=True)

# Sort by median age for clearer analysis
median_age_by_perf.sort_values(by='Median Age', ascending=False, inplace=True)

display(
    median_age_by_perf.style
    .set_caption("Median Age by Performance Score")
    .format({'Median Age': '{:.2f}'))
)
```

<pandas.io.formats.style.Styler at 0x1c0f57bc310>

SCENARIO 3: MODE OF ENGAGEMENT SURVEY BY POSITION

Purpose: This step identifies the most frequently occurring engagement survey score for each position. This highlights the dominant sentiment (positive or negative) for employees in each role.

Rationale:The mode is ideal here because we're interested in the most common experience or perception, not the average. It helps HR detect positions where employees consistently feel disengaged or highly engaged.

```
[15]: # Define a custom function to calculate the mode of a series
def calculate_mode(series):
    """Return the mode of a Series or NaN if the mode cannot be determined."""
    mode_result = stats.mode(series, keepdims=True)
    return mode_result.mode[0] if mode_result.count[0] > 0 else np.nan

# Apply the mode function for each Position
mode_engagement_by_pos = df.groupby('Position')['EngagementSurvey'].
    ↪apply(calculate_mode).reset_index()
mode_engagement_by_pos.rename(columns={'EngagementSurvey': 'Mode of Engagement_
    ↪Survey'}, inplace=True)

# Sort for readability
mode_engagement_by_pos.sort_values(by='Mode of Engagement Survey',
    ↪ascending=False, inplace=True)
```

```
display(
    mode_engagement_by_pos.style
    .set_caption("Mode of Engagement Survey by Position")
    .format({'Mode of Engagement Survey': '{:.2f}'))
)
```

<pandas.io.formats.style.Styler at 0x1c0f637d210>

OVERALL CENTRAL TENDENCIES (MEAN, MEDIAN, MODE)

Purpose: This step calculates the three core measures of central tendency, mean, median, and mode for important numerical attributes in the HR dataset.

Rationale:

- Mean reflects the average value (sensitive to outliers).
- Median gives the middle value (resistant to outliers).
- Mode reveals the most common value (especially useful for categorical-like numeric fields).

Using all three provides a more complete picture of the data distribution.

```
[16]: # Define a function to safely calculate the mode of a pandas Series
def get_mode(series):
    """
    Return the mode of a Series or NaN if the Series is empty.

    Parameters:
        series (pd.Series): The input pandas Series.

    Returns:
        float: The mode of the Series, or NaN if the Series is empty.
    """
    mode_result = stats.mode(series, keepdims=True)
    return mode_result.mode[0] if mode_result.count[0] > 0 else np.nan

# List of key attributes to analyze
attributes = ['Salary', 'Age', 'EngagementSurvey']

# Calculate mean, median, and mode for each attribute
central_tendencies = pd.DataFrame({
    'Attribute': attributes,
    'Mean': [df[attr].mean() for attr in attributes], # Compute mean for each
    'Median': [df[attr].median() for attr in attributes], # Compute median for
})
```

```

    'Mode': [get_mode(df[attr]) for attr in attributes] # Compute mode using
    ↪ the get_mode function
}).round(2)

# Display the results
display(
    central_tendencies.style
    .set_caption("Overall Central Tendencies")
    .format({
        'Mean': '{:,.2f}',
        'Median': '{:,.2f}',
        'Mode': '{:,.2f}'
    })
)

```

<pandas.io.formats.style.Styler at 0x1c0f57724d0>

0.1.4 Task 2: Measures of Spread and Dispersion

SCENARIO 1: SALARY DISPERSION BY DEPARTMENT

Purpose: This analysis explores how salaries vary across different departments using three key dispersion metrics:

- **Range:** The gap between the highest and lowest salary
- **Variance:** The average of squared salary differences from the mean (sensitive to outliers)
- **Standard Deviation (Std. Dev):** The spread of salaries around the mean (in the same units as salary)

Rationale: While averages are useful, they can hide disparities. Dispersion metrics reveal whether departments have consistent pay scales or wide gaps important for equity and fairness reviews.

```

[17]: # Group by department and calculate salary spread metrics
stats = df.groupby('Department')['Salary'].agg(
    Range=lambda x: x.max() - x.min(),
    Variance='var',
    StdDev='std'
).reset_index()

# Rename columns for clarity
stats = stats.rename(columns={
    'Range': 'Salary Range',
    'Variance': 'Salary Variance',
    'StdDev': 'Salary Std. Dev'
})

```

```

})

# Round values for better readability
stats = stats.round(2)

# Sort by highest standard deviation
stats.sort_values(by='Salary Std. Dev', ascending=False, inplace=True)

# Print output
display(
    stats.style
        .format('{:,.2f}', subset=stats.columns[1:])
        .set_caption("Salary Dispersion by Department")
)

```

<pandas.io.formats.style.Styler at 0x1c0f5d96410>

SCENARIO 2: AGE DISPERSION BY PERFORMANCE SCORE

Purpose: This analysis evaluates how employee ages vary across different performance scores using:

- **Range:** The spread between the youngest and oldest employee
- **Variance:** The average of squared differences from the mean age
- **Standard Deviation:** The typical amount by which ages deviate from the mean

Rationale: Identifying how age spreads across performance categories may suggest generational trends, experience differences, or even unconscious bias. HR can use this to better align coaching, training, or policy efforts.

```

[18]: # Group by performance score and calculate age dispersion metrics
age_stats = df.groupby('PerformanceScore')['Age'].agg(
    Range=lambda x: x.max() - x.min(),
    Variance='var',
    StdDev='std'
).reset_index()

# Rename columns for clarity
age_stats = age_stats.rename(columns={
    'Range': 'Age Range',
    'Variance': 'Age Variance',
    'StdDev': 'Age Std. Dev'
})

# Round values
age_stats = age_stats.round(2)

```



```

# Sort by standard deviation for easy insight
age_stats.sort_values(by='Age Std. Dev', ascending=False, inplace=True)

# Print output
display(
    age_stats.style
        .format('{:,.2f}', subset=age_stats.columns[1:])
        .set_caption("Age Dispersion by Performance Score")
)

```

<pandas.io.formats.style.Styler at 0x1c0f64657d0>

SCENARIO 3: TENURE DISPERSION BY DEPARTMENT

Purpose: This step explores how employee tenure varies across departments using:

- **Range:** The difference between the longest and shortest serving employees
- **Variance:** The spread of tenure values squared
- **Standard Deviation:** The average deviation from the mean tenure (in years)

Rationale: Understanding tenure spread reveals departmental stability, turnover risks, and succession planning needs. Departments with unusually high tenure variance might require deeper investigation (e.g., onboarding effectiveness, promotion delays).

```

[19]: # Group by department and calculate tenure dispersion metrics
tenure_stats = df.groupby('Department')['Tenure'].agg(
    Range=lambda x: x.max() - x.min(),
    Variance='var',
    StdDev='std'
).reset_index()

# Rename columns for clarity
tenure_stats = tenure_stats.rename(columns={
    'Range': 'Tenure Range',
    'Variance': 'Tenure Variance',
    'StdDev': 'Tenure Std. Dev'
})

# Round results for readability
tenure_stats = tenure_stats.round(2)

# Sort by Tenure Std. Dev for better comparison
tenure_stats.sort_values(by='Tenure Std. Dev', ascending=False, inplace=True)

# Print output

```

```
display(
    tenure_stats.style
        .format('{:,.2f}', subset=tenure_stats.columns[1:])
        .set_caption("Tenure Dispersion by Department")
)
```

<pandas.io.formats.style.Styler at 0x1c0f64657d0>

SCENARIO 4: SPECIAL PROJECTS DISPERSION BY POSITION

Purpose: This section measures the variation in how many special projects employees are involved in, grouped by their positions.

We use:

- Range: Difference between highest and lowest project count
- Variance: Measure of spread squared
- Standard Deviation: Interpretable spread around the mean

Rationale: Disparities in project assignments could reflect unequal opportunities, unfair workloads, or differences in leadership expectations. Understanding which roles experience high variability helps management better allocate tasks and recognize high contributors.

```
[20]: # Group by position and calculate project count dispersion
projects_stats = df.groupby('Position')['SpecialProjectsCount'].agg(
    Range=lambda x: x.max() - x.min(),
    Variance='var',
    StdDev='std'
).reset_index()

# Rename columns for clarity
projects_stats = projects_stats.rename(columns={
    'Range': 'Projects Range',
    'Variance': 'Projects Variance',
    'StdDev': 'Projects Std. Dev'
})

# Round values
projects_stats = projects_stats.round(2)

# Sort to highlight positions with highest variability
projects_stats.sort_values(by='Projects Std. Dev', ascending=False,
    inplace=True)

# Print output
```

```

display(
    projects_stats.style
        .format('{:,.2f}', subset=projects_stats.columns[1:])
        .set_caption("Special Projects Dispersion by Position")
        .set_table_styles([{'selector': 'th', 'props': [('background-color', '□', '↪'#f0f0f0'), ('font-weight', 'bold')]}])
)

```

<pandas.io.formats.style.Styler at 0x1c0f7675290>

SCENARIO 5: LATENESS DISPERSION BY PERFORMANCE SCORE

Purpose: This section examines how lateness behavior (number of days late in the past 30 days) varies across different performance scores.

We compute:

- Range: Difference between the most and least days late
- Variance: Squared deviations from the mean lateness
- Standard Deviation: Typical deviation in lateness

Rationale: Performance is often linked to punctuality. Analyzing lateness dispersion helps HR identify patterns of reliability within performance categories, informing coaching and attendance policies.

```

[21]: # Group by performance score and calculate lateness dispersion metrics
lateness_stats = df.groupby('PerformanceScore')['DaysLateLast30'].agg(
    Range=lambda x: x.max() - x.min(),
    Variance='var',
    StdDev='std'
).reset_index()

# Rename for clarity
lateness_stats = lateness_stats.rename(columns={
    'Range': 'Days Late Range',
    'Variance': 'Days Late Variance',
    'StdDev': 'Days Late Std. Dev'
})

# Round values for better readability
lateness_stats = lateness_stats.round(2)

# Sort by standard deviation to find inconsistent performers
lateness_stats.sort_values(by='Days Late Std. Dev', ascending=False, '□', '↪inplace=True)

```

```

# Print output
display(
    lateness_stats.style
        .format('{:,.2f}', subset=lateness_stats.columns[1:])
        .set_caption("Lateness Dispersion by Performance Score")
)

```

<pandas.io.formats.style.Styler at 0x1c0f64657d0>

SUMMARY OF SPREAD METRICS

Purpose: This summary provides a side-by-side comparison of dispersion statistics Range, Variance, and Standard Deviation for core workforce variables like Salary, Age, Tenure, SpecialProjectsCount, and DaysLateLast30.

Rationale: This enables quick comparison of consistency vs variability across key performance, behavior, and demographic metrics. High dispersion in any metric may indicate inconsistency, outliers, or areas for policy review (e.g., pay equity, punctuality).

```

[22]: # Define the numeric columns to summarize
spread_metrics = {
    'Salary': df['Salary'],
    'Age': df['Age'],
    'Tenure': df['Tenure'],
    'SpecialProjectsCount': df['SpecialProjectsCount'],
    'DaysLateLast30': df['DaysLateLast30']
}

# Create a summary DataFrame for dispersion measures
summary_data = pd.DataFrame({
    'Attribute': list(spread_metrics.keys()),
    'Range': [series.max() - series.min() for series in spread_metrics.
        ↪ values()],
    'Variance': [series.var() for series in spread_metrics.values()],
    'Std. Dev': [series.std() for series in spread_metrics.values()]
}).round(2)

# Print output
display(
    summary_data.style
        .set_caption("Summary of Spread Metrics")
        .format({'Range': '{:,.2f}', 'Variance': '{:,.2f}', 'Std. Dev': '{:,.2f}'})
)

```

<pandas.io.formats.style.Styler at 0x1c0f7641650>

0.1.5 Task 3: Confidence Intervals

SCENARIO 1: 95% CONFIDENCE INTERVALS FOR SALARY BY DEPARTMENT

Purpose: This section estimates the range in which the true mean salary likely falls for each department, using a 95% confidence level.

Rationale: Confidence intervals add statistical rigor to our analysis. They show the level of uncertainty in the estimated means. For instance, smaller intervals suggest greater consistency and reliability, while wider intervals may indicate outliers or small sample sizes.

```
[23]: import pandas as pd
import numpy as np
from scipy import stats

# Define a function to calculate the confidence interval for a data series
def confidence_interval(data, confidence=0.95):
    """Calculate the confidence interval for a numeric Series."""
    data = data.dropna()
    n = len(data)
    if n < 2:
        return np.nan, np.nan
    mean = np.mean(data)
    std_err = stats.sem(data.to_numpy(), nan_policy='omit')
    margin = std_err * stats.t.ppf((1 + confidence) / 2, n - 1)
    return mean - margin, mean + margin

# Apply the confidence interval function to the 'Salary' column grouped by
↳ 'Department'
salary_ci_by_department = df.groupby('Department')['Salary'].
↳ apply(confidence_interval).reset_index()

# Format the result into readable string intervals
salary_ci_by_department['Confidence Interval'] =
↳ salary_ci_by_department['Salary'].apply(
    lambda x: f"({x[0]:.2f}, {x[1]:.2f})" if isinstance(x, tuple) and not
↳ any(np.isnan(x)) else "NaN"
)

# Keep only relevant columns
salary_ci_by_department = salary_ci_by_department[['Department', 'Confidence
↳ Interval']]

# Print output
display(
    salary_ci_by_department.style
```

```

        .set_caption("95% Confidence Intervals for Salary by Department")
        .set_table_styles([{'selector': 'th', 'props': [('background-color',
↪ '#f0f0f0'), ('font-weight', 'bold')]}])
    )

```

<pandas.io.formats.style.Styler at 0x1c0f65b04d0>

SCENARIO 2: 95% CONFIDENCE INTERVALS FOR ENGAGEMENT SURVEY BY PERFORMANCE SCORE

Purpose: This section estimates the confidence intervals for average EngagementSurvey scores within each performance score category.

Rationale: It helps HR understand how consistent engagement levels are among top vs low performers. Tighter intervals suggest a uniform experience, while wider ones suggest more variation in perception, which may signal management or communication gaps.

```

[24]: # Define the function to compute confidence intervals with error handling
def confidence_interval(data, confidence=0.95):
    """Calculate the confidence interval for a numeric Series with safety
↪ checks."""
    data = data.dropna()
    n = len(data)
    if n < 2:
        return np.nan, np.nan
    try:
        mean = np.mean(data)
        std_err = stats.sem(data.to_numpy(), nan_policy='omit')
        margin = std_err * stats.t.ppf((1 + confidence) / 2, n - 1)
        return mean - margin, mean + margin
    except Exception as e:
        print(f"Error calculating SEM: {e}")
        return np.nan, np.nan

# Apply to EngagementSurvey grouped by PerformanceScore
engagement_ci_by_performance = df.
↪ groupby('PerformanceScore')['EngagementSurvey'].apply(confidence_interval).
↪ reset_index()

# Format CI as a readable string
engagement_ci_by_performance['Confidence Interval'] =
↪ engagement_ci_by_performance['EngagementSurvey'].apply(
    lambda x: f"({x[0]:.2f}, {x[1]:.2f})" if isinstance(x, tuple) and not
↪ any(np.isnan(x)) else "NaN"
)

# Keep only relevant columns

```

```

engagement_ci_by_performance =
    engagement_ci_by_performance[['PerformanceScore', 'Confidence Interval']]

# Print output
display(
    engagement_ci_by_performance.style
        .set_caption("95% Confidence Intervals for Engagement Survey by Performance_
Score")
        .set_table_styles([{'selector': 'th', 'props': [('background-color',
    '#f0f0f0'), ('font-weight', 'bold')]}])
)

```

<pandas.io.formats.style.Styler at 0x1c0f7698c10>

GENERAL 95% CONFIDENCE INTERVALS FOR SALARY & ENGAGEMENT SURVEY

Purpose: This section provides overall 95% confidence intervals for key workforce metrics Salary and EngagementSurvey. These intervals give us a statistically valid range where the true population mean likely lies.

Rationale: Such insights help leadership understand the reliability and spread of compensation and engagement data without needing to segment by department or performance.

[25]: *# Define function to calculate confidence intervals with built-in safety*

```

def confidence_interval(data, confidence=0.95):
    """Calculate confidence interval with error handling."""
    data = data.dropna()
    n = len(data)
    if n < 2:
        return np.nan, np.nan
    try:
        mean = np.mean(data)
        std_err = stats.sem(data.to_numpy(), nan_policy='omit')
        margin = std_err * stats.t.ppf((1 + confidence) / 2, n - 1)
        return mean - margin, mean + margin
    except Exception as e:
        print(f"Error calculating SEM: {e}")
        return np.nan, np.nan

# Calculate CIs for Salary and EngagementSurvey
salary_ci = confidence_interval(df['Salary'])
engagement_ci = confidence_interval(df['EngagementSurvey'])

# Format into a DataFrame
general_ci = pd.DataFrame({
    'Attribute': ['Salary', 'EngagementSurvey'],

```

```

        'Confidence Interval': [
            f"({salary_ci[0]:,.2f}, {salary_ci[1]:,.2f})" if isinstance(salary_ci, tuple) and not any(np.isnan(salary_ci)) else "NaN",
            f"({engagement_ci[0]:,.2f}, {engagement_ci[1]:,.2f})" if isinstance(engagement_ci, tuple) and not any(np.isnan(engagement_ci)) else "NaN"
        ]
    })

# Print output
display(
    general_ci.style
    .set_caption("General 95% Confidence Intervals")
    .set_table_styles([{'selector': 'th', 'props': [('background-color', '#f0f0f0'), ('font-weight', 'bold')]}])
)

```

<pandas.io.formats.style.Styler at 0x1c0f76bc990>

0.1.6 Task 4: Hypothesis Testing

HYPOTHESIS TESTING 1: SALARY DIFFERENCE BY PERFORMANCE SCORE

Purpose: This section tests whether there's a statistically significant difference in average salary between employees rated as “Fully Meets” vs “Exceeds” performance expectations.

Rationale: Rather than assume higher performers are paid more, we test it using an independent t-test (Welch's t-test), which does not assume equal variances.

Hypotheses:

- Null Hypothesis (H₀): There is no significant difference in salary between the two performance groups.
- Alternative Hypothesis (H_a): There is a significant difference in salary between the groups.

```

[26]: # Filter salary values by performance groups
fully_meets = df.loc[df['PerformanceScore'] == 'Fully Meets', 'Salary']
exceeds = df.loc[df['PerformanceScore'] == 'Exceeds', 'Salary']

# Perform Welch's t-test
t_stat, p_val = stats.ttest_ind(fully_meets, exceeds, equal_var=False)

# Interpret the result based on p-value
conclusion = (

```



```

    "Reject the null hypothesis. Significant difference in salary based on_
    ↪performance score."
    if p_val < 0.05 else
    "Fail to reject the null hypothesis. No significant salary difference based_
    ↪on performance score."
)

# Create a DataFrame to summarize the t-test results
t_test_results = pd.DataFrame({
    "Comparison": ["Fully Meets vs Exceeds"],
    "T-statistic": [round(t_stat, 2)],
    "P-value": [round(p_val, 4)],
    "Conclusion": [conclusion]
})

# Print output
display(
    t_test_results.style
    .set_caption("T-Test Results: Salary by Performance Score")
    .set_table_styles([{'selector': 'th', 'props': [('background-color',_
    ↪'#f0f0f0'), ('font-weight', 'bold')]}])
)

```

<pandas.io.formats.style.Styler at 0x1c0f769bf50>

HYPOTHESIS TESTING 2: ENGAGEMENT SURVEY SCORES ACROSS DEPARTMENTS

Purpose: This section uses a one-way ANOVA (Analysis of Variance) test to determine whether the average EngagementSurvey score differs significantly between departments.

Rationale: While t-tests compare two groups, ANOVA is used when comparing three or more. It tests the hypothesis that all group means are equal, which helps HR identify departments with significantly different engagement levels.

Hypotheses:

- Null Hypothesis (H₀): There is no difference in EngagementSurvey means across departments.
- Alternative Hypothesis (H_a): At least one department has a significantly different mean.

```

[27]: # Group engagement scores by department and remove NaNs
engagement_by_department = [
    group['EngagementSurvey'].dropna()
    for name, group in df.groupby('Department')
]

```

```

]

# Perform one-way ANOVA
f_stat, p_value = stats.f_oneway(*engagement_by_department)

# Interpret the result
conclusion = (
    "Reject the null hypothesis. There is a significant difference in_
    ↪EngagementSurvey scores across departments."
    if p_value < 0.05 else
    "Fail to reject the null hypothesis. No significant difference in_
    ↪EngagementSurvey scores across departments."
)

# Organize the results
anova_results = pd.DataFrame({
    "F-statistic": [round(f_stat, 2)],
    "P-value": [round(p_value, 4)],
    "Conclusion": [conclusion]
})

# Print output
display(
    anova_results.style
    .set_caption("ANOVA Results: Engagement Survey Scores by Department")
    .set_table_styles([{'selector': 'th', 'props': [('background-color',
    ↪'#f0f0f0'), ('font-weight', 'bold')]}])
)

```

<pandas.io.formats.style.Styler at 0x1c0f76bea90>

HYPOTHESIS TESTING 3: SPECIAL PROJECTS COUNT BY POSITION

Purpose: This section uses one-way ANOVA to test whether the average number of special projects assigned varies significantly between employee positions.

Rationale: This helps identify whether some roles consistently receive more project opportunities than others. Such a pattern could reflect differences in responsibility, management decisions, or project availability.

Hypotheses:

- Null Hypothesis (H₀): There is no difference in SpecialProjectsCount means across positions.
- Alternative Hypothesis (H_a): At least one position has a significantly different project count.

```

[28]: # Group special project counts by position, removing NaN values
projects_by_position = [
    group['SpecialProjectsCount'].dropna()
    for name, group in df.groupby('Position')
]

# Perform one-way ANOVA
f_stat, p_value = stats.f_oneway(*projects_by_position)

# Interpret the result
conclusion = (
    "Reject the null hypothesis. There is a significant difference in_
↳SpecialProjectsCount across positions."
    if p_value < 0.05 else
    "Fail to reject the null hypothesis. No significant difference in_
↳SpecialProjectsCount across positions."
)

# Create results table
anova_results = pd.DataFrame({
    "F-statistic": [round(f_stat, 2)],
    "P-value": [round(p_value, 4)],
    "Conclusion": [conclusion]
})

# Print output
display(
    anova_results.style
        .set_caption("ANOVA Results: Special Projects Count by Position")
        .set_table_styles([{'selector': 'th', 'props': [('background-color', '
↳#f0f0f0'), ('font-weight', 'bold')]}])
)

```

```
<pandas.io.formats.style.Styler at 0x1c0f768f210>
```